

RBAC Design Approach

GHAS Demo Orchestration Platform

Security Architecture Document • v1.0

Prepared by	Hicks — Security Architect
Requested by	Saurabh
Classification	Internal — Confidential
Platform	TypeScript / Express / Single-page HTML
Endpoints	39+ API routes across 7 route groups
Frontend	10-tab presenter console
MCP Server	16 tools (Jira, Risk, Governance, License)

Table of Contents

- 1** Executive Summary
- 2** Role Definitions
 - 2.1** Security Admin
 - 2.2** Security Analyst
 - 2.3** Developer
 - 2.4** Team Lead
 - 2.5** Compliance Officer / Auditor
 - 2.6** Demo Operator
 - 2.7** Viewer (Read-Only)
 - 2.8** System Admin
- 3** Permission Matrix
- 4** Technical Architecture
 - 4.1** JWT Authentication
 - 4.2** Express Middleware
 - 4.3** Frontend Tab Gating
 - 4.4** Token Structure & Session Management
- 5** Implementation Approach
- 6** Security Considerations
- 7** Proposed File Layout

1 Executive Summary

The GHAS Demo Orchestration Platform aggregates highly sensitive security telemetry — live CVE feeds, zero-day intelligence, PII citizen records, SBOM exports, governance audit trails, and direct Jira integration — into a single Express server serving 39+ API endpoints and a 10-tab HTML console. Today, **every endpoint is unauthenticated and unguarded**. Any network-reachable client can trigger live demo incidents, read PII, export SBOMs, or modify governance policies.

This document proposes a **Role-Based Access Control (RBAC)** layer that enforces the principle of least privilege across every route group and frontend surface. The design is JWT-based, middleware-driven, and phased for incremental rollout with zero downtime.

Business Justification

- **Regulatory Compliance** — PII endpoints (SSN, DOB, address) require access control under GDPR, CCPA, and SOC 2 Type II.
- **Separation of Duties** — Demo operators should not access production CVE intelligence; auditors should not modify policies.
- **Audit Readiness** — Every access control decision must be logged for compliance evidence.
- **Blast Radius Containment** — A compromised demo operator token must not grant zero-day intel or Jira write access.
- **Enterprise Adoption** — Customers evaluating GHAS expect the demo platform itself to demonstrate security best practices.
- **Insurance & Liability** — Unrestricted PII access creates legal exposure; RBAC is a baseline control.

2 Role Definitions

Eight roles are defined, ordered from most to least privileged. Each role is additive — higher roles inherit the read permissions of lower roles where noted.

2.1 Security Admin

Persona: CISO, Head of AppSec

Full platform access including policy management, user/role administration, and all security intelligence surfaces. This is the superuser role.

Business Justification: The security leader needs unrestricted access to configure policies, manage risk exceptions, and respond to zero-day events without delay.

■ Permitted Actions	■ Denied Actions
All API endpoints (CRUD)	Cannot bypass audit logging
Manage users and role assignments	Cannot delete audit trail entries
Configure governance policies	Cannot disable RBAC system
Approve/reject risk exceptions	
Export SBOMs and audit trails	
Run and manage live demos	
Access raw PII (with audit log)	

2.2 Security Analyst

Persona: SOC Analyst, Vuln Management Engineer

Read access to all security intelligence — CVEs, zero-day, trends, governance. Can create and manage Jira tickets for vulnerability tracking. Cannot modify platform policies.

Business Justification: Analysts triage vulnerabilities daily and need comprehensive read access plus the ability to create tracking tickets, but must not alter security policies.

■ Permitted Actions	■ Denied Actions
Read CVE feed, zero-day intel, trends	Cannot modify governance policies
Search and filter vulnerabilities	Cannot approve risk exceptions
Create/update Jira tickets	Cannot manage users
View governance checks and audit trail	Cannot run live demos
View redacted PII records	Cannot access raw PII
Export reports (read-only)	

2.3 Developer

Persona: Software Engineer, SRE

View CVEs relevant to their codebase, check governance status on their PRs, view vulnerability trends. Scoped to development-relevant security data.

Business Justification: Developers must see how security findings affect their code and PRs, enabling shift-left remediation without exposing sensitive operational intelligence.

■ Permitted Actions	■ Denied Actions
View CVE feed (read-only)	Cannot access zero-day intel
View governance checks on own PRs	Cannot access PII data
View vulnerability trends	Cannot create risk exceptions
View Jira tickets (own team)	Cannot run demos
View license compliance status	Cannot manage Jira tickets (other teams)
	Cannot access audit trail

2.4 Team Lead

Persona: Engineering Manager, Staff Engineer

Developer permissions plus team management — approve risk exceptions for their team, manage Jira tickets, view team-level metrics.

Business Justification: Leads are accountable for their team's security posture and need approval authority for risk exceptions and visibility into team remediation metrics.

■ Permitted Actions	■ Denied Actions
All Developer permissions	Cannot modify governance policies
Approve risk exceptions (own team)	Cannot manage users globally
Manage Jira tickets (own team)	Cannot access PII data
View team MTTR and remediation rates	Cannot run live demos
View SBOM for team dependencies	Cannot approve exceptions for other teams

2.5 Compliance Officer / Auditor

Persona: GRC Analyst, External Auditor

Read-only access to audit trails, license compliance, governance reports, and SBOM exports. Designed for compliance evidence gathering.

Business Justification: Auditors need verifiable evidence of security controls without the ability to modify any data, ensuring audit independence.

■ Permitted Actions	■ Denied Actions
Read audit trail (full history)	Cannot modify any data
Export governance reports	Cannot create Jira tickets
View license compliance data	Cannot run demos

Export SBOM	Cannot access raw security intel
View risk exception history	Cannot manage users or policies
View redacted PII access logs	

2.6 Demo Operator

Persona: Solutions Engineer, Sales Engineer

Can run live demo scenarios, manage demo branches, and trigger workflows. Isolated from production security data to prevent accidental exposure during customer demos.

Business Justification: Demo operators present to external audiences and must be sandboxed to demo functionality only — leaking zero-day intel or PII during a demo would be catastrophic.

■ Permitted Actions	■ Denied Actions
Run/stop demo scenarios	Cannot access CVE feed
Create demo branches	Cannot access zero-day intel
Trigger demo workflows	Cannot access PII data
View demo status and results	Cannot access Jira
Access GHAS Showcase tab	Cannot modify governance
View Business vs Enterprise comparison	Cannot access audit trail

2.7 Viewer (Read-Only)

Persona: Executive, Stakeholder, New Hire

Dashboard-only access with no action capabilities. Can view high-level security posture but cannot drill into sensitive data or perform any mutations.

Business Justification: Executives and stakeholders need a security posture overview without the risk of accidental data modification or sensitive data exposure.

■ Permitted Actions	■ Denied Actions
View vulnerability trend dashboards	Cannot access raw CVE data
View GHAS Showcase information	Cannot access zero-day intel
View Business vs Enterprise comparison	Cannot access PII
View Copilot Security overview	Cannot run demos
	Cannot create tickets
	Cannot export any data

2.8 System Admin

Persona: Platform Engineer, DevOps Lead

User and role management, system configuration, health monitoring. No access to security intelligence data — pure operational role.

Business Justification: Separation of duties requires that the person managing user accounts does not have access to the security data those accounts protect.

■ Permitted Actions	■ Denied Actions
Manage user accounts (CRUD)	Cannot access CVE/zero-day data
Assign/revoke roles	Cannot access PII
View system health and logs	Cannot modify governance policies
Configure rate limits and session policies	Cannot run demos
Manage API keys and tokens	Cannot access Jira bridge
	Cannot approve risk exceptions

3 Permission Matrix

The matrix below maps each role to each resource with granular CRUD permissions. Legend: **FULL** = Create/Read/Update/Delete, **READ** = Read-only, **NONE** = No access, **SCOPED** = Conditional access (own team/own PRs).

Role	Demo Ops	PII Data	CVE Feed	Zero-Day	Trends	Governance	Risk Exc.	License	Jira	Audit Trail	User Mgmt	Policy Cfg
Sec Admin	FULL	FULL	FULL	FULL	FULL	FULL	FULL	FULL	FULL	READ	FULL	FULL
Sec Analyst	NONE	READ	READ	READ	READ	READ	NONE	READ	FULL	READ	NONE	NONE
Developer	NONE	NONE	READ	NONE	READ	SCOPED	NONE	READ	SCOPED	NONE	NONE	NONE
Team Lead	NONE	NONE	READ	NONE	READ	SCOPED	SCOPED	READ	SCOPED	NONE	NONE	NONE
Auditor	NONE	READ	NONE	NONE	NONE	READ	READ	READ	NONE	READ	NONE	NONE
Demo Op	FULL	NONE	NONE	NONE	NONE	NONE	NONE	NONE	NONE	NONE	NONE	NONE
Viewer	NONE	NONE	NONE	NONE	READ	NONE	NONE	NONE	NONE	NONE	NONE	NONE
Sys Admin	NONE	NONE	NONE	NONE	NONE	NONE	NONE	NONE	NONE	READ	FULL	READ

Route Group → Role Mapping (API Paths)

Route Group	Endpoints	Allowed Roles	Permission
/api/demos/*	8 endpoints	Security Admin, Demo Operator	FULL
/api/pii-demo/*	6 endpoints	Security Admin (full), Analyst/Auditor (redacted)	SCOPED
/api/cves/*	5 endpoints	Security Admin, Security Analyst	READ+
/api/zero-day/*	5 endpoints	Security Admin, Security Analyst	READ
/api/trends/*	6 endpoints	Sec Admin, Analyst, Developer, Team Lead, Viewer	READ
/api/governance/*	5 endpoints	Sec Admin (full), Analyst/Auditor (read), Dev/Lead (scoped)	SCOPED
/api/jira/*	4 endpoints	Sec Admin, Analyst (full), Team Lead (scoped)	SCOPED

4 Technical Architecture

4.1 JWT Authentication

All authentication uses JSON Web Tokens (RFC 7519) signed with RS256 (asymmetric). Tokens are issued by a dedicated auth service and validated by Express middleware on every request.

Token Payload Structure

```
{
  "sub": "user_id_uuid",
  "name": "Jane Doe",
  "email": "jane@corp.com",
  "role": "security_analyst",
  "permissions": ["cve:read", "zeroday:read", "jira:write", "trends:read",
    "governance:read", "audit:read", "pii:read_redacted"],
  "team_id": "appsec-team-01",
  "iat": 1720000000,
  "exp": 1720003600,
  "iss": "ghas-demo-platform",
  "jti": "unique-token-id"
}
```

Token Lifecycle

- **Access Token** — 1 hour expiry, stored in httpOnly secure cookie, used for API auth.
- **Refresh Token** — 7 day expiry, stored in httpOnly secure cookie (separate path), rotated on use.
- **Token Revocation** — JTI-based denylist in Redis/memory for immediate revocation on logout or role change.
- **Key Rotation** — RS256 key pairs rotated every 90 days; JWKS endpoint serves public keys.

4.2 Express Middleware Architecture

Three middleware layers execute in order on every request:

Layer	File	Responsibility
1. authenticate	src/middleware/auth.ts	Extract JWT from cookie/header, verify signature and expiry, attach user context to req.user
2. authorize	src/middleware/rbac.ts	Check req.user.permissions against route's required permission. Return 403 if denied.
3. auditAccess	src/middleware/audit.ts	Log the access decision (allow/deny) with user, route, timestamp, IP to audit trail.

Route Guard Example

```

import { requirePermission } from "../middleware/rbac";

// CVE routes – require cve:read
router.get("/api/cves/latest", requirePermission("cve:read"), cveController.getLatest);
router.get("/api/cves/search", requirePermission("cve:read"), cveController.search);

// Governance – scoped to own team for developers
router.get("/api/governance/checks",
  requirePermission("governance:read", { scopeToTeam: true }),
  govController.getChecks
);

// Risk exceptions – team leads can approve their own team only
router.post("/api/governance/exceptions/:id/approve",
  requirePermission("risk_exception:approve", { scopeToTeam: true }),
  govController.approveException
);

```

4.3 Frontend Tab Visibility Gating

The server injects a **user context object** into the HTML response. Each tab's visibility is controlled by checking `user.permissions` client-side. This is a UX convenience only — all security enforcement happens server-side via middleware.

Tab	Required Permission	Visible To
0. Live Demos	demo:execute	Security Admin, Demo Operator
1. GHAS Showcase	showcase:read	All authenticated roles
2. Biz vs Enterprise	plans:read	All authenticated roles
3. GitHub vs Competitors	competitive:read	Security Admin, Demo Operator, Viewer
4. Copilot Security	copilot:read	All authenticated roles
5. CVE Feed	cve:read	Security Admin, Security Analyst
6. Zero-Day Intel	zeroday:read	Security Admin, Security Analyst
7. Vuln Trends	trends:read	Sec Admin, Analyst, Developer, Team Lead, Viewer
8. Jira Bridge	jira:read	Sec Admin, Analyst, Team Lead
9. Governance	governance:read	Sec Admin, Analyst, Auditor, Dev (scoped), Lead (scoped)

4.4 Session Management Strategy

- **Stateless JWT** — No server-side session store required. All claims embedded in token.

- **Cookie Transport** — Tokens sent via httpOnly, Secure, SameSite=Strict cookies to prevent XSS/CSRF.
- **Refresh Flow** — On 401 response, frontend silently calls /api/auth/refresh. If refresh fails, redirect to login.
- **Concurrent Sessions** — Max 3 active sessions per user. New login invalidates oldest session via JTI denylist.
- **Idle Timeout** — 30 minutes of inactivity triggers client-side logout. Configurable per role.
- **Role Change Propagation** — On role update, all active JTIs for that user are added to denylist, forcing re-auth.

5 Implementation Approach

Four-phase rollout, each independently deployable and backward-compatible.

Phase 1 — Auth Service + JWT Middleware

Duration: 2–3 weeks

- Implement `src/services/auth-service.ts` (login, register, token issue/refresh/revoke)
- Create `src/middleware/auth.ts` (JWT verification, user context injection)
- Create `src/middleware/rbac.ts` (permission checking with role → permission mapping)
- Define role and permission enums in `src/models/rbac.ts`
- Add user table to SQLite database (id, email, password_hash, role, team_id)
- Implement `/api/auth/*` routes (login, logout, refresh, me)
- Add RS256 key pair generation and JWKS endpoint
- Write integration tests for auth flow

Phase 2 — Route-Level Guards

Duration: 2 weeks

- Add `requirePermission()` middleware to all 39+ API endpoints
- Implement team-scoped permissions for governance and Jira routes
- Add PII access tier: raw (admin) vs redacted (analyst/auditor) vs denied (others)
- Implement rate limiting per role (higher limits for admin/analyst)
- Add audit logging middleware to record every access decision
- Update API error responses with standardized 401/403 payloads
- Create permission integration tests for each route group

Phase 3 — Frontend Role-Aware Rendering

Duration: 1–2 weeks

- Inject user context into `index.html` via server-side template variable
- Gate tab visibility based on `user.permissions` array
- Disable action buttons (Run Demo, Create Ticket, Approve Exception) per role
- Add login/logout UI to the presenter console
- Show current user role badge in header
- Add graceful 403 handling (friendly message, not raw error)
- Ensure all security enforcement remains server-side (frontend is UX only)

Phase 4 — Admin Panel + User Management

Duration: 2 weeks

- Build admin panel (Tab 10 or separate `/admin` route)
- Implement user CRUD: create, update role, deactivate, delete
- Add role assignment UI with team scoping
- Implement active session management (view, revoke)
- Add API key management for MCP server tools
- Build audit log viewer with filtering and export
- Add OIDC/SAML SSO integration point for enterprise deployment

Total Estimated Effort: 7–9 weeks for a single engineer, or 4–5 weeks with two engineers working in parallel (Phase 1+3, Phase 2+4).

6 Security Considerations

Principle of Least Privilege

- Every role starts with zero permissions; access is explicitly granted.
- Developers cannot see zero-day intelligence — only CVEs and trends relevant to their work.
- Demo operators are sandboxed to demo routes only — no production security data.
- Auditors get read-only access — cannot modify any data, ensuring audit independence.

Separation of Duties

- System Admins manage users but cannot access security data.
- Security Admins access data but user management is a shared responsibility with System Admins.
- The person who creates a risk exception cannot approve it (four-eyes principle).
- Demo operators and security analysts operate in completely isolated permission domains.

Token Security

- **httpOnly cookies** — Tokens never accessible via JavaScript, preventing XSS token theft.
- **Secure flag** — Cookies only sent over HTTPS.
- **SameSite=Strict** — Prevents CSRF by blocking cross-origin cookie transmission.
- **Short access token expiry (1 hour)** — Limits window of token compromise.
- **Refresh token rotation** — Each refresh issues a new token and invalidates the old one.
- **JTI denylist** — Enables immediate token revocation without waiting for expiry.

Audit Logging

- Every permission check (allow and deny) is logged with user, route, timestamp, source IP.
- PII access is logged to a separate audit trail with the data fields accessed.
- Audit logs are append-only — no role can delete or modify entries.
- Logs are structured for SIEM ingestion (JSON format, consistent schema).

Rate Limiting per Role

- Security Admin: 1000 req/min (operational needs during incidents).
- Analyst: 300 req/min (normal triage workflow).
- Developer / Team Lead: 120 req/min (code review workflow).
- Viewer: 60 req/min (dashboard polling).
- Unauthenticated: 10 req/min (brute-force protection).

Defense in Depth

- Frontend tab hiding is UX only — never the sole access control mechanism.
- API middleware enforces permissions regardless of how the request arrives.

- MCP server tools validate permissions using the same RBAC system.
- Database queries are parameterized — RBAC does not replace SQL injection prevention.
- Network-level controls (IP allowlisting) complement application-level RBAC.

7 Proposed File Layout

All new files integrate into the existing Express project structure.

File Path	Purpose
src/models/rbac.ts	Role enum, Permission enum, RolePermissionMap, User interface, TokenPayload interface
src/services/auth-service.ts	Login, register, token issue (RS256), refresh, revoke, password hashing (bcrypt)
src/middleware/auth.ts	JWT verification, cookie extraction, user context injection into req.user
src/middleware/rbac.ts	requirePermission(perm, opts?) factory — returns Express middleware that checks req.user.permissions
src/middleware/audit.ts	Access decision logging middleware — logs allow/deny with full context
src/middleware/rate-limiter.ts	Role-aware rate limiting using sliding window counters
src/api/auth-routes.ts	POST /api/auth/login, /logout, /refresh, GET /api/auth/me, /api/auth/jwks
src/api/admin-routes.ts	User CRUD, role assignment, session management, API key management
src/security/token-denylist.ts	In-memory (or Redis) JTI denylist for token revocation
src/security/key-manager.ts	RS256 key pair generation, rotation, JWKS endpoint support
data/users.db	SQLite database for user accounts (id, email, password_hash, role, team_id, created_at)
public/js/auth.js	Client-side auth utilities: login form, token refresh, tab gating, role badge
.env	Add: JWT_PRIVATE_KEY_PATH, JWT_PUBLIC_KEY_PATH, JWT_EXPIRY, REFRESH_EXPIRY

End of Document

Prepared by Hicks, Security Architect — for internal review only.